

AI MINI PROJECT — TECHNICAL REPORT

BudgetIQ

An AI-Powered Personal Budget Management System — Full-Stack Web Application

REPORT DATE

March 27, 2026

PROJECT TYPE

AI Mini Project

STATUS

 Production Deployed

FRONTEND URL

ai-budgetiq.vercel.app

BACKEND URL

Render (budgetiq-backend)

DATABASE

Supabase (PostgreSQL)

Prepared by: Vijaya Gowtham | Academic Year 2025–26

BudgetIQ

1

Executive Summary

BudgetIQ is a full-stack, AI-assisted personal budget management web application designed to help individuals track income, monitor expenses, set spending limits, and receive real-time financial insights — all through a modern, responsive interface.

The project demonstrates end-to-end software engineering skills, covering frontend development with React.js, backend API development with Python Flask, cloud database integration with Supabase (PostgreSQL), JWT-based authentication, and full production deployment on Vercel and Render.

The "AI" component provides rule-based financial insights and an interactive chat assistant that analyzes a user's financial data and gives personalised savings advice, spending breakdowns, and goal tracking — making it suitable as an AI mini-project submission.



PRIMARY GOAL

Help users take control of their personal finances with intelligent, data-driven insights and goal tracking.



AI COMPONENT

Rule-based AI chat assistant that analyses income, expenses, and category breakdowns to give personalised advice.



DEPLOYMENT

Frontend on Vercel, backend on Render, PostgreSQL database on Supabase — fully production-ready.

Problem Statement & Objectives

2.1 Problem Statement

Many individuals struggle to manage their personal finances effectively due to a lack of accessible, easy-to-use tools that combine expense tracking with intelligent recommendations. Traditional spreadsheet-based approaches are tedious, error-prone and provide no automated insight.

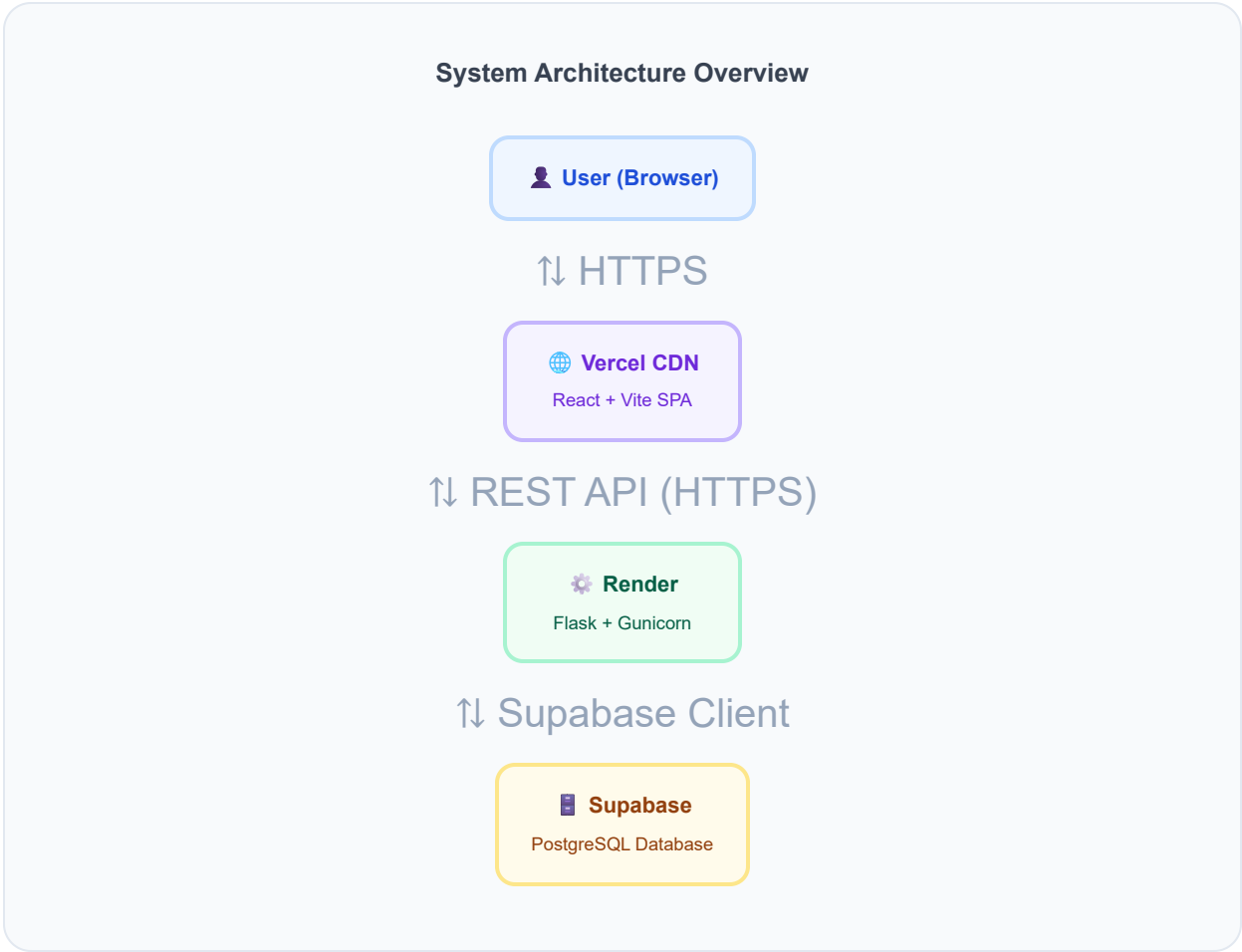
2.2 Project Objectives

1. Build a **secure, multi-user web application** for tracking income and expenses in Indian Rupees (₹).
2. Implement an **AI-powered financial assistant** that can analyse data and answer questions in natural language.
3. Provide a **visual dashboard** with charts showing spending trends over the last 6 months.
4. Allow users to **set budget goals** per spending category and track savings targets.
5. **Deploy the full stack** to production cloud platforms with proper environment configuration.
6. Ensure the UI is **fully responsive** for both desktop and mobile devices.

3

System Architecture

BudgetIQ follows a classic **three-tier architecture**: a React SPA as the presentation layer, a Flask REST API as the application layer, and Supabase (PostgreSQL) as the data layer.



3.1 Component Responsibilities

LAYER	TECHNOLOGY	RESPONSIBILITY
Frontend (SPA)	React 18, Vite, TailwindCSS	Renders all UI, handles routing, manages authentication state in localStorage, calls backend APIs.
Backend (API)	Python 3, Flask 3, Gunicorn	Exposes REST endpoints, validates JWT tokens, implements business logic and the AI service, interfaces with Supabase.
Database	Supabase (PostgreSQL)	Persists user accounts, income records, expense records. Provides real-time capabilities and hosted infrastructure.
Auth	PyJWT, Werkzeug	Password hashing (PBKDF2-SHA256), JWT token issuance and verification on every protected route.

Deployment	Vercel, Render	Vercel hosts the static frontend build; Render hosts the Python backend with auto-deploy from GitHub.
------------	----------------	---

4

Technology Stack

4.1 Frontend

 React 18.3

 Vite 5.3

 TailwindCSS 3.4

 React Router DOM 6.25

 Recharts 2.12

 Lucide React

 clsx + tailwind-merge

4.2 Backend

 Python 3.x

 Flask ≥3.0

 Flask-CORS 4.0

 PyJWT ≥2.8

 Werkzeug ≥3.0

 Supabase-py ≥2.0

 python-dotenv

 Gunicorn ≥21.2

4.3 Infrastructure & DevOps

SERVICE	PURPOSE	DETAILS
Supabase	Database & BaaS	Managed PostgreSQL, row-level data isolation, REST client SDK
Vercel	Frontend Hosting	Automatic HTTPS, global CDN, SPA rewrite rules via <code>vercel.json</code>
Render	Backend Hosting	Auto-deploy from GitHub, Gunicorn WSGI server, env var management via <code>render.yaml</code>
GitHub	Version Control	Source of truth for CI/CD pipelines to Vercel and Render

Application Features



PUBLIC LANDING PAGE

A polished marketing page showcasing features, an interactive demo mode, and calls-to-action for signup/login. Fully animated with dark-theme design.



AUTHENTICATION

JWT-based secure login and registration. Passwords hashed with PBKDF2-SHA256. 7-day token expiry with auto-redirect on session expiry.



DASHBOARD OVERVIEW

Real-time stat cards for total balance, income, and expenses. Recharts area chart showing income vs. expense for the last 6 months. Recent transactions list.



INCOME TRACKING

Add income entries with source, amount (₹), and date. View chronological list of all income records. Delete entries with ownership verification.



EXPENSE TRACKING

Log expenses by category (Groceries, Transport, Entertainment, etc.), description, amount, and date. CSV export functionality for external use.



AI FINANCIAL ASSISTANT

Interactive chat interface powered by rule-based AI. Answers questions about spending, savings rates, top categories, and financial goals using real user data.



GOALS & BUDGET LIMITS

Set a personal savings target and track progress via progress bars. Configure per-category monthly spending limits with visual alerts when near or over budget.



REPORTS

Dedicated reports page for deeper financial analysis and data visualization. Supports CSV export of expense data for external spreadsheet analysis.



PROFILE MANAGEMENT



GUEST DEMO MODE

Update name, email, and upload a profile avatar (base64 stored in Supabase). Account deletion with cascading cleanup of all user data.

A fully interactive demo page with pre-loaded sample data, allowing prospective users to explore all features before registering.

6

Backend API Design

The Flask backend exposes a RESTful API under the `/api` prefix. All data-access routes are protected by a JWT middleware decorator (`@token_required`). The application is structured using the **Blueprint pattern** for modularity.

6.1 Project Structure

```
backend/ ├── app.py # Entry point - creates & runs Flask app ├── config.py # Config class
(Supabase, JWT keys) ├── requirements.txt # Python dependencies ├── Procfile # Gunicorn start
command for Render └── app/ ├── __init__.py # create_app() factory, Blueprint registration ├──
extensions.py # Supabase client singleton (get_supabase) ├── routes/ ├── auth_routes.py #
/api/auth/* - register, login, profile ├── income_routes.py # /api/income/* - add, list,
delete ├── expense_routes.py# /api/expense/* - add, list, delete ├── dashboard_routes.py #
/api/dashboard/* - summary, monthly, ai_chat ├── services/ ├── ai_service.py # Rule-based
financial insight generator ├── utils/ ├── middleware.py # token_required JWT decorator
```

6.2 API Endpoints Reference

Authentication — `/api/auth`

POST	/api/auth/register	Register new user (email, password, full_name)
POST	/api/auth/login	Login, returns JWT access_token (7-day expiry)
POST	/api/auth/logout	Logout (client-side token removal)
PUT	/api/auth/profile	🔒 Update name, email, avatar
DEL	/api/auth/profile	🔒 Delete account and all associated data

Income — `/api/income`

POST	/api/income/add	🔒 Add income record (source, amount, date)
GET	/api/income/list	🔒 List all income sorted by date desc
DEL	/api/income/{id}	🔒 Delete income record (ownership verified)

Expenses — `/api/expense`

POST	/api/expense/add	🔒 Add expense (category, amount, date, desc)
------	------------------	--

GET	/api/expense/list	🔒 List all expenses sorted by date desc
DEL	/api/expense/{id}	🔒 Delete expense (ownership verified)

Dashboard — /api/dashboard

GET	/api/dashboard/summary	🔒 Total income, expenses, balance + AI insight
GET	/api/dashboard/monthly	🔒 Income & expense totals for last 6 months
POST	/api/dashboard/ai_chat	🔒 AI assistant response to user financial query

🔒 **PROTECTED** These routes require a valid `Authorization: Bearer <token>` header

7

AI Module — Financial Intelligence

BudgetIQ includes a rule-based AI engine implemented in `app/services/ai_service.py` and extended in the `/api/dashboard/ai_chat` route. It analyses a user's real financial data to generate personalised insights.

7.1 AI Insight Generator (`analyze_finances`)

Called on every dashboard load to produce an automatic one-line financial health message:

```
def analyze_finances(total_income, total_expense): if total_expense > total_income: return "⚠️
Warning: overspending detected..." savings_rate = ((total_income - total_expense) /
total_income) * 100 if savings_rate >= 20: return f"✅ Great job! Saving {savings_rate:.1f}% of
income." elif savings_rate > 0: return f"💡 Try to reach the 20% savings target."
```

7.2 AI Chat Assistant

The `/api/dashboard/ai_chat` endpoint processes natural language queries from the user and returns a contextual response using real-time data from Supabase:

DETECTED INTENT	KEYWORDS	RESPONSE TYPE
Spending Reduction	reduce, cut, save, spending, dining	Identifies highest spending category, advises weekly budget cap
Goal Tracking	goal, track, target, savings rate	Reports current savings %, compares vs 20% benchmark
Financial Overview	analyze, breakdown, summary, overview	Calls <code>analyze_finances</code> + lists top 3 spending categories
Income Query	income, earn, salary	Reports total income and annualised projection
General Query	(anything else)	Snapshot: income, expenses, balance, savings rate

The AI module aggregates user-specific data (income totals, expense categories) from live database records for each query — ensuring responses are always personalised and up-to-date.

8

Frontend Architecture

8.1 Project Structure

```
src/ |— App.jsx # Root router – public + auth + protected routes |— main.jsx # React DOM entry point |— index.css # Global styles & TailwindCSS base |— pages/ | |— Landing.jsx # Public marketing/hero page | |— Demo.jsx # Interactive guest demo mode | |— Login.jsx # Login form | |— Signup.jsx # Registration form | |— Dashboard.jsx # Overview with charts & AI insight | |— Income.jsx # Income CRUD UI | |— Expenses.jsx # Expense CRUD + CSV export | |— AIInsights.jsx# Interactive AI chat page | |— Goals.jsx # Budget limits & savings goal tracker | |— Reports.jsx # Financial reports & analytics | |— Profile.jsx # User profile & account settings |— components/ |— layout/ # AuthLayout, MainLayout (sidebar + topbar) |— ui/ # Card, Button, Input, Modal, Skeleton, Toast |— context/ | |— ToastContext.jsx # Global toast notification system |— services/ | |— api.js # Fetch wrapper for all API calls (auto-auth) |— utils/ | |— currency.js # fmt() helper for ₹ formatting
```

8.2 Routing Architecture

ROUTE	COMPONENT	ACCESS
/	Landing	PUBLIC
/demo	Demo	PUBLIC
/login	Login	AUTH LAYOUT
/signup	Signup	AUTH LAYOUT
/dashboard	Dashboard	PROTECTED
/income	Income	PROTECTED
/expenses	Expenses	PROTECTED
/insights	AllInsights	PROTECTED
/goals	Goals	PROTECTED
/reports	Reports	PROTECTED
/profile	Profile	PROTECTED

8.3 State Management & API Layer

State is managed locally within each page component using React hooks (`useState` , `useEffect`). A shared `api.js` service encapsulates all HTTP communication with automatic JWT injection, error handling, and 401-redirect logic. Toast notifications are provided globally via React Context.

9

Database Design

The application uses **Supabase (PostgreSQL)** as its database. Three primary tables store all application data.

9.1 Schema Overview

Table: `users`

COLUMN	TYPE	DESCRIPTION
<code>id</code>	BIGINT (PK, auto-increment)	Unique user identifier
<code>email</code>	TEXT (unique)	User's email address (login credential)
<code>password_hash</code>	TEXT	PBKDF2-SHA256 hashed password
<code>full_name</code>	TEXT	User's display name
<code>avatar</code>	TEXT (nullable)	Base64-encoded profile picture
<code>created_at</code>	TIMESTAMPTZ	Auto-set on insert

Table: `income`

COLUMN	TYPE	DESCRIPTION
<code>id</code>	BIGINT (PK)	Record identifier
<code>user_id</code>	BIGINT (FK → users)	Owner of this record
<code>amount</code>	NUMERIC	Income amount in ₹
<code>source</code>	TEXT	Source of income (e.g. "Salary")
<code>date</code>	DATE	Date of income

Table: `expenses`

COLUMN	TYPE	DESCRIPTION
<code>id</code>	BIGINT (PK)	Record identifier
<code>user_id</code>	BIGINT (FK → users)	Owner of this record

amount	NUMERIC	Expense amount in ₹
category	TEXT	Spending category (Groceries, Transport, etc.)
description	TEXT	Optional description of the expense
date	DATE	Date of expense

9.2 Data Isolation

Every query filters by `user_id = current_user_id` (extracted from the JWT). Ownership is explicitly verified before any delete operation, preventing unauthorised data access.

Security Implementation



PASSWORD SECURITY

All passwords are hashed using **PBKDF2-SHA256** via Werkzeug before storage. Plaintext passwords are never stored or logged.



JWT AUTHENTICATION

Stateless authentication using JSON Web Tokens (HS256). Tokens expire after 7 days. Every protected route validates the token via `@token_required`.



CORS POLICY

Flask-CORS is configured to allow cross-origin requests only from known frontend domains, preventing unauthorised browser-based API access.



ENVIRONMENT VARIABLES

All secrets (Supabase URL, Service Key, JWT Secret) are stored as environment variables — never hardcoded. `.env` files are git-ignored.



INPUT VALIDATION

All API endpoints validate required fields, data types, and positive amount values before touching the database. Appropriate HTTP status codes are returned on errors.



DATA OWNERSHIP

Delete operations verify `user_id` ownership before proceeding. A user cannot access or modify another user's records.

11

Deployment & DevOps

11.1 Frontend Deployment — Vercel

The React application is built with **Vite** and deployed to Vercel's global CDN. SPA routing is handled by a rewrite rule in `vercel.json` :

```
{ "rewrites": [{ "source": "/(.*)", "destination": "/" } ] }
```

The `VITE_API_URL` environment variable points to the Render backend URL, set in Vercel's project settings.

11.2 Backend Deployment — Render

The Flask API is deployed via `render.yaml` with Gunicorn as the production WSGI server:

```
# render.yaml
services:
  - type: web
    name: budgetiq-backend
    runtime: python
    rootDir: backend
    buildCommand: pip install -r requirements.txt
    startCommand: gunicorn app:app
```

Sensitive environment variables (Supabase credentials, JWT secret) are configured in the Render dashboard — not in the YAML file.

11.3 Deployment Flow

STEP	ACTION	OUTCOME
1	Developer pushes code to GitHub	Triggers Vercel & Render auto-deploys
2	Vercel runs <code>vite build</code>	Bundles React SPA to <code>/dist</code> folder
3	Vercel deploys <code>/dist</code> to CDN	Frontend live at <code>ai-budgetiq.vercel.app</code>
4	Render runs <code>pip install</code>	Python deps installed in virtualenv
5	Render starts <code>gunicorn app:app</code>	REST API live and accepting connections
6	Health check: <code>GET /health</code>	Returns <code>{"status":"healthy"}</code>

12

Testing & Validation

12.1 Manual Testing

TEST CASE	EXPECTED RESULT	STATUS
User Registration with valid data	201 Created, user record in DB	✓ PASS
Duplicate email registration	400 Bad Request with error message	✓ PASS
Login with correct credentials	200 OK, JWT token returned	✓ PASS
Login with wrong password	401 Unauthorized	✓ PASS
Add income record	201, record visible in income list	✓ PASS
Add expense record	201, record visible in expense list	✓ PASS
Dashboard summary loads	Correct totals calculated	✓ PASS
AI chat responds to savings query	Returns savings rate and advice	✓ PASS
Goals page — set budget limit	Progress bar updates, alert on overspend	✓ PASS
Access protected route without token	401 Unauthorized, redirect to login	✓ PASS
Delete another user's expense	404 Not Found (ownership check)	✓ PASS
CSV export of expenses	Downloadable CSV file generated	✓ PASS

12.2 Error Handling

The backend validates all inputs and returns consistent JSON error responses. HTTPS errors trigger appropriate UI messages via the toast notification system. The frontend's `parseResponse` function handles both JSON and non-JSON server responses gracefully.

13

Challenges & Solutions

CHALLENGE	SOLUTION ADOPTED
CORS errors in production Frontend on Vercel couldn't reach Flask API on Render.	Configured <code>Flask-CORS</code> with <code>origins="*"</code> for the <code>/api/</code> namespace and verified <code>VITE_API_URL</code> was set correctly on Vercel.
JWT token expiry handling Expired tokens caused silent failures in the UI.	Added a global 401 handler in <code>api.js</code> that clears <code>localStorage</code> and redirects to <code>/login</code> automatically.
Database migration to Supabase Originally used SQLite; migrated to Supabase for production scalability.	Replaced SQLAlchemy ORM with the Supabase Python SDK. Implemented a singleton <code>get_supabase()</code> client factory in <code>extensions.py</code> .
Mobile responsiveness Sidebar navigation caused overflow on small screens.	Implemented a hamburger menu toggle for mobile viewports and applied responsive Tailwind classes (<code>md:</code> , <code>lg:</code>) throughout all layouts and pages.
Real-time AI insights Providing personalised advice without a remote LLM API.	Built a rule-based AI service that analyses actual user data from Supabase and uses keyword-matching intent detection to return contextual, data-driven responses.

Future Enhancements



LLM INTEGRATION

Replace rule-based AI with a real LLM (e.g. Google Gemini API or OpenAI GPT) for true natural language understanding and more nuanced financial advice.



MOBILE APP

Build a React Native companion app for on-the-go expense logging with push notifications for budget alerts.



EMAIL ALERTS

Send automated email notifications when a user exceeds a budget limit or is close to their savings goal deadline.



BANK INTEGRATION

Integrate with open banking APIs (e.g., Plaid or Indian UPI data feeds) to automatically import transactions.



ADVANCED ANALYTICS

Add predictive spending forecasts, year-over-year comparisons, and exportable PDF financial reports.



MULTI-CURRENCY SUPPORT

Allow users to switch between currencies with live exchange rates for users travelling internationally.

Conclusion

BudgetIQ successfully delivers a production-ready, AI-assisted personal finance management platform that demonstrates a complete mastery of full-stack web development — from database design and secure REST APIs to a polished, responsive React frontend.

The project achieves all its stated objectives: a secure multi-user system, a functional AI financial assistant, rich data visualisations, goal tracking, and a full cloud deployment. The system currently serves as a live, accessible application at **ai-budgetiq.vercel.app**.

The AI component, while rule-based, demonstrates the key principles of applied AI — intent detection, data-driven personalisation, and contextual response generation — making it an appropriate and meaningful "AI Mini Project" submission.

The project showcases key software engineering practices including:

- Secure authentication with password hashing and JWT tokens
- RESTful API design with Blueprint-based modular architecture
- Cloud-native deployment with CI/CD via GitHub → Vercel/Render
- Responsive component-based frontend with reusable UI primitives
- Proper separation of concerns across all layers of the stack



Project Successfully Completed & Deployed

BudgetIQ — AI-Powered Personal Budget Management System

React 18 · Flask 3 · Supabase · Vercel · Render · JWT · Recharts